

LilyPond_transpose
Python 版 利用・実装解説

smat1957

2026 年 9 月 6 日

本書は、LilyPond で記述された楽譜を指定音程だけ移調する LilyPond_transpose の Python 版を説明する。第 I 部ではコマンドラインからの使い方と実例を示し、第 II 部では実際のプログラムソースを参照しながら、字句解析、音楽理論計算、相対音高の解決、移調、LilyPond 文字列への復元という処理を解説する。本プログラムは完全な LilyPond パーサーではなく、バッハ無伴奏チェロ組曲をギター用に移調・整形する実用範囲を主な対象としている。

目次

第 I 部	使い方と使用例	5
第 1 章	概要	7
1.1	目的	7
1.2	必要な環境	7
第 2 章	基本操作	9
2.1	コマンド形式	9
2.2	音名と臨時記号	9
2.3	オクターブ指示	9
第 3 章	使用例	11
3.1	C-Dur から A-Dur へ移調する	11
3.2	C-Dur から D-Dur へ移調する	11
3.3	G-Dur から D-Dur へ完全 5 度上げる	11
3.4	変換結果を LilyPond で確認する	11
第 4 章	対応範囲と注意事項	13
第 II 部	プログラムソースと処理内容	15
第 5 章	全体構成	17
5.1	処理の流れ	17
第 6 章	エントリーポイント	19
6.1	main.py の役割	19
第 7 章	Token による中間表現	23
7.1	tokens.py の役割	23
第 8 章	字句解析	27
8.1	tokenizer.py の役割	27
第 9 章	音名と半音の理論計算	35
9.1	music.theory.py の役割	35
第 10 章	絶対音高とオクターブ	41
10.1	pitch.py の役割	41

第 11 章	相対音高の解決	45
11.1	relative.py の役割	45
第 12 章	Token 列の移調	49
12.1	transpose.py の役割	49
第 13 章	LilyPond 文字列への復元	53
13.1	writer.py の役割	53
第 14 章	保守と検証	55
14.1	変更時に確認する事項	55

第 I 部

使い方と使用例

第 1 章

概要

1.1 目的

Python 版は、標準入力から LilyPond ソースを読み、移調後の LilyPond ソースを標準出力へ書き出すコマンドラインプログラムである。入力ファイルを直接上書きしないため、変換前の楽譜を残したまま別ファイルとして結果を生成できる。

主な処理対象は次のとおりである。

- LilyPond 音名と臨時記号
- `\relative` ブロックとオクターブ記号
- 単音と和音<...>
- `\key` による調指定
- 複数声部を含む並列ブロック
- コメント、音価、演奏記号など移調対象外の文字列の保持

1.2 必要な環境

移調処理には Python 3 を使用する。結果を譜面として確認するには LilyPond も必要である。リポジトリの代表的な構成は次のとおりである。

```
LilyPond_transpose/  
|-- python/      Python 移調プログラム  
|-- lilyp/       入力・出力 LilyPond ソース  
|-- pdf/         生成した PDF  
|-- swift/       Swift 版  
`-- gen*.sh      曲別の生成スクリプト
```


第 2 章

基本操作

2.1 コマンド形式

```
cd python
python3 main.py SRC DST < input.ly > output.ly
```

SRC は移調元の基準音高、DST は移調先の基準音高である。単なる調名ではなく、オクターブ指示も含む音高として解釈される。プログラムは

$$\text{shift} = \text{pitch}(DST) - \text{pitch}(SRC) \quad (2.1)$$

によって半音移動量を求める。

2.2 音名と臨時記号

音名は LilyPond 形式で指定する。代表例を表 2.1 に示す。

表 2.1: 代表的な音名

表記	意味	備考
c, d, e	自然音	C, D, E
cis, fis, gis	シャープ	C♯, F♯, G♯
des, ges	フラット	D♭, G♭
es, as, bes	フラット	E♭, A♭, B♭

通常用いる長調の主音として、ces, ges, des, as, es, bes, f, c, g, d, a, e, b, fis, cis を指定できる。入力中の \major または \minor は維持されるため、同じ引数形式を長調と短調の両方に使用する。

2.3 オクターブ指示

ASCII アポストロフィー' は 1 個につき 1 オクターブ上、カンマ, は 1 個につき 1 オクターブ下を表す。例えば次の二つは移調後の音名文字が同じでも方向が異なる。

指定	半音移動量	意味
g から d	−5	完全 4 度下
g から d'	+7	完全 5 度上

g から d,	-17	1 オクターブと完全 4 度下
c から c',	+12	1 オクターブ上
c から c,	-12	1 オクターブ下

bash や zsh からアポストロフィーを渡す場合は、引数全体をダブルクォートで囲む方法が分かりやすい。

```
python3 main.py g "d'" < input.ly > output.ly
```

次のようにアポストロフィーだけをバックスラッシュでエスケープしても同じ結果になる。

```
python3 main.py g d\' < input.ly > output.ly
```

バックスラッシュはシェルにアポストロフィーを解釈させないためのものであり、Python へ渡す音高文字列の一部ではない。アプリや HTTP API など、シェルを介さず引数を渡す場合は `d'` をそのまま使用する。アポストロフィーとカンマを混在させた `d'`, は受け付けない。

第 3 章

使用例

3.1 C-Dur から A-Dur へ移調する

```
cd python
python3 main.py c a \
  < ../lilyp/cello/cello1009.ly \
  > ../lilyp/guitar/guitar1009A.ly
```

c から a は 3 半音下である。入力中の `\key c \major` は `\key a \major` へ変換され、音符と和音も同じ音程だけ移調される。

3.2 C-Dur から D-Dur へ移調する

```
cd python
python3 main.py c d \
  < ../lilyp/cello/cello1009.ly \
  > ../lilyp/guitar/guitar1009D.ly
```

この指定は 2 半音上への移調である。

3.3 G-Dur から D-Dur へ完全 5 度上げる

```
cd python
python3 main.py g "d'" \
  < ../lilyp/cello/cello1007.ly \
  > ../lilyp/guitar/guitar1007.ly
```

d' を指定することで、同じ D-Dur でも 5 半音下ではなく 7 半音上を選択する。

3.4 変換結果を LilyPond で確認する

移調後は必ず LilyPond でコンパイルし、構文、音高、調号、オクターブ、運指、譜面の読みやすさを確認する。

```
cd ../lilyp
lilypond BWV1009PreludeA.ly
mv BWV1009PreludeA.pdf ../pdf/
```

曲別の `gen1007.sh`、`gen1009A.sh` などを使えば、移調、PDF 生成、既存結果との比較をまとめて実行できる。

第 4 章

対応範囲と注意事項

本プログラムは LilyPond の全構文を扱う完全なパーサーではない。次の点に注意する。

- 複雑な Scheme 式や独自マクロの内部は正しく解析できない場合がある。
- 移調元引数と入力中の調号が一致するかは自動確認しない。
- 移調後の綴りに通常範囲を超える臨時記号が必要な場合はエラーになることがある。
- `\relative` と多声部の組合せは実際の楽譜で確認する必要がある。
- 変換結果を完成譜とせず、必ず LilyPond によるコンパイルと目視確認を行う。

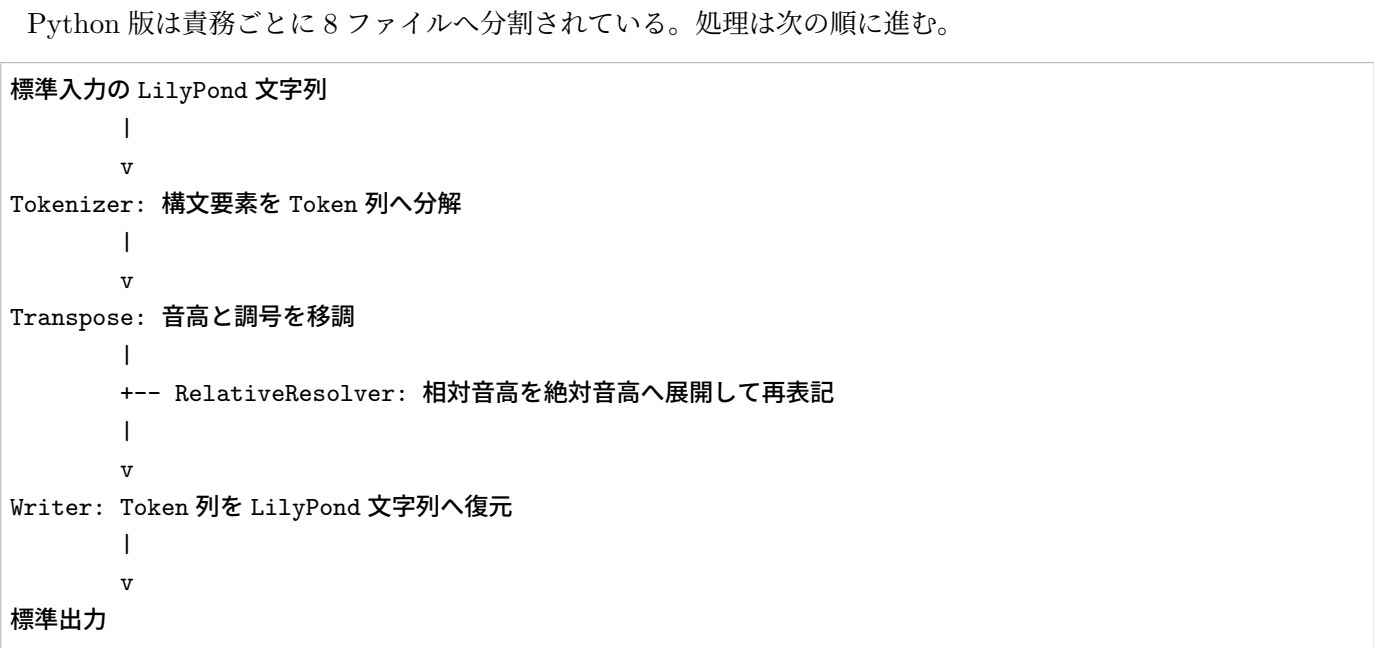
第II部

プログラムソースと処理内容

第 5 章

全体構成

5.1 処理の流れ



ファイル	主な責務
main.py	引数、標準入力、処理全体の制御
tokens.py	Token データ型の定義
tokenizer.py	LilyPond 文字列から Token 列への変換
music_theory.py	音名、半音値、異名同音、綴りの計算
pitch.py	音高文字列、オクターブ、MIDI 値の相互変換
relative.py	LilyPond 相対音高の解決と再生成
transpose.py	Token 木の走査と移調処理の振り分け
writer.py	Token 列から LilyPond 文字列への復元

第 6 章

エントリーポイント

6.1 main.py の役割

main.py は二つの引数を検査し、parse_pitch() で半音移動量を、letter_shift_between() で音名文字の移動量を求める。この二種類の移動量を分けることで、同じ半音値を持つ音でも、移調音程に沿った音名文字を選ぶ。

標準入力全体を Tokenizer へ渡し、transpose_tokens() で Token 列を変更した後、各 Token を write_token() で標準出力へ書き出す。

```

1  '''
2  main.py
3  エントリーポイント
4
5  tokens.py
6  定義 Token
7
8  tokenizer.py
9  文字列 → Token
10
11 music_theory.py
12 音名・半音理論
13
14 pitch.py
15   ・変換 PitchPosMIDI
16
17 relative.py
18 解決 Relative
19
20 transpose.py
21 走査 Token
22
23 writer.py
24  Token → LilyPond
25  '''
26 # main.py
27 #
28 # LilyPond 移調プログラムのエントリーポイント。
29 #
30 # 処理の流れ:
31 #
32 #   標準入力
33 #   ↓
34 #   Tokenize
35 #   ↓
36 #   移調
37 #   ↓
38 #   文字列へ復元 LilyPond
39 #   ↓
40 #   標準出力
41 #
42 # 使用例:
43 #
44 #   python main.py c a \
45 #       < cello.ly \
46 #       > guitar.ly
47 #
48 # この例では
49 #
50 #   調基準 c → 調基準 a
51 #
52 #   の移調を行う。
53
54 import sys

```

```

55
56 from tokenizer import Tokenizer
57 from transpose import (
58     parse_pitch,
59     letter_shift_between,
60     transpose_tokens,
61 )
62 from writer import write_token
63
64
65 def main():
66     """
67     メイン処理。
68
69     コマンドライン引数:
70
71         argv[1]
72         移調元音名
73
74         argv[2]
75         移調先音名
76
77     例:
78
79         python main.py c a
80     """
81
82     # 引数チェック
83     if len(sys.argv) != 3:
84
85         print(
86             "usage: python main.py SRC DST",
87             file=sys.stderr
88         )
89
90         print(
91             "example: python main.py c a",
92             file=sys.stderr
93         )
94
95         sys.exit(1)
96
97     # 移調元音名
98     src = sys.argv[1]
99
100    # 移調先音名
101    dst = sys.argv[2]
102
103    # 半音移動量を計算する。
104    #
105    # 例:
106    #
107    #     c → a
108    #
109    #     なら -3
110    #
111    shift = (
112        parse_pitch(dst)
113        -
114        parse_pitch(src)
115    )
116
117    # 音名文字の移動量を計算する。
118    #
119    # 例:
120    #
121    #     c → a
122    #
123    #     なら
124    #
125    #     c d e f g a
126    #
127    #     なので 5
128    #
129    letter_shift = letter_shift_between(
130        src,
131        dst
132    )
133
134    # ソースを標準入力から読む。LilyPond
135    text = sys.stdin.read()
136
137    # 列へ変換する。Token
138    tokens = Tokenizer(
139        text
140    ).tokenize()
141
142    # 列を移調する。Token
143    transpose_tokens(

```

```
144         tokens,
145         shift,
146         letter_shift
147     )
148
149     # ソースへ戻して標準出力へ書く。LilyPond
150     for t in tokens:
151         sys.stdout.write(
152             write_token(t)
153         )
154
155
156 # スクリプトとして実行された場合のみ main() を呼ぶ。
157 if __name__ == "__main__":
158     main()
```

Listing 6.1 エントリーポイント main.py

第 7 章

Token による中間表現

7.1 tokens.py の役割

文字列を直接置換すると、音名と変数名、コメント、コマンドなどを区別できない。そのため、プログラムはいったん入力を Token 列へ変換する。NoteToken は音名、オクターブ記号、音価などの後続文字列を分離して保持する。RelativeBlock、ChordToken、ParallelBlock は子 Token を持ち、LilyPond 構文の入れ子を表現する。

```

1  # tokens.py
2  #
3  # LilyPond ソースを解析した結果を保持する
4  # Token クラス群を定義する。
5  #
6  # tokenizer.py は文字列を Token に変換し、
7  # transpose.py は Token を移調し、
8  # writer.py は Token を LilyPond ソースへ戻す。
9  #
10 # ここで定義するクラスは、
11 # LilyPond の構文要素を表現するための
12 # データ構造である。
13
14 from dataclasses import dataclass
15
16
17 @dataclass
18 class Token:
19     """
20     全 Token の基底クラス。
21
22     実際には直接生成せず、
23     各種 Token クラスの共通親として使う。
24     """
25     pass
26
27
28 @dataclass
29 class RelativeBlock(Token):
30     """
31     \\relative ブロック。
32
33     例:
34
35         \\relative c' {
36             c d e
37         }
38
39     anchor
40         c'
41
42     tokens
43         ブロック内部の Token 列
44     """
45     anchor: str
46     tokens: list
47
48
49 @dataclass
50 class RawToken(Token):
51     """
52     そのまま出力する文字列。
53
54     主に空白や改行、
55     Token 化対象でない文字列を保持する。
56     """
57     text: str
58 
```

```

59
60 @dataclass
61 class CommentToken(Token):
62     """
63     LilyPond コメント。
64
65     例:
66
67         % Prelude
68
69     % 以降改行直前までを保持する。
70     """
71     text: str
72
73
74 @dataclass
75 class CommandToken(Token):
76     """
77     LilyPond コマンド。
78
79     例:
80
81         \\clef
82         \\time
83         \\major
84         \\tempo
85
86     コマンド名全体を保持する。
87     """
88     text: str
89
90
91 @dataclass
92 class NoteToken(Token):
93     """
94     単音。
95
96     例:
97
98         c'8
99         fis,16~
100         bes4(
101
102     note
103     音名
104
105     octave
106     オクターブ記号 LilyPond
107     ( ' や , )
108
109     suffix
110     音価やスラー等の後続文字列
111     """
112     note: str
113     octave: str
114     suffix: str
115
116
117 @dataclass
118 class ChordToken(Token):
119     """
120     和音。
121
122     例:
123
124         <c e g>4
125
126     items
127     和音内部の Token 列
128
129     suffix
130     > の後ろに続く音価等
131     """
132     items: list
133     suffix: str
134
135
136 @dataclass
137 class SymbolToken(Token):
138     """
139     単独の記号。
140
141     例:
142
143         {
144         }
145         <<
146         >>
147

```



```

148     構文解析に必要な記号を保持する。
149     """
150     text: str
151
152
153 @dataclass
154 class ParallelBlock(Token):
155     """
156     << ... >> 形式の並列ブロック。
157
158     例:
159
160         <<
161             { voice1 }
162             { voice2 }
163         >>
164
165     voices
166     各声部の Token 列のリスト
167     """
168     voices: list
169
170
171 @dataclass
172 class KeyToken(Token):
173     """
174     \\key コマンド。
175
176     例:
177
178         \\key c \\major
179         \\key a \\minor
180
181     key
182     調名
183
184     mode
185     major / minor
186     """
187     key: str
188     mode: str

```

Listing 7.1 Token 定義 tokens.py

第 8 章

字句解析

8.1 tokenizer.py の役割

Tokenizer は入力位置を進めながら、コメント、コマンド、音符、和音、調指定、\relative、並列ブロックを読み取る。音名一覧は長い順に並べ、beses を b として途中まで誤認しないようにしている。

音符は音名、' や, からなるオクターブ記号、空白までの suffix へ分ける。suffix を保持することで、音価、タイ、スラー、装飾などを移調後も書き戻せる。

read_relative() は外側の波括弧に対応する範囲を切り出し、その内部を新しい Tokenizer で再帰的に解析する。同様に和音や複数声部も子 Token 列として保持する。

```
1 # tokenizer.py
2 #
3 # LilyPond のソース文字列を読み取り、
4 # tokens.py で定義した Token 列へ変換する。
5 #
6 # 例:
7 #
8 #   c'8 d e
9 #
10 # を
11 #
12 #   NoteToken("c", "'", "8")
13 #   RawToken(" ")
14 #   NoteToken("d", "", "")
15 #   ...
16 #
17 # のような構造に分解する。
18
19 # まず Token 定義
20 from tokens import *
21
22 # 音名定義
23 #
24 # LilyPond の音名を長い順に並べる。
25 # 例:
26 #
27 #   beses
28 #   bes
29 #   b
30 #
31 # のような音名では、短い b を先に読むと誤認識するため、
32 # 必ず長い音名から判定する。
33 NOTE_NAMES = [
34     "beses", "ases", "ees", "eis",
35     "ces", "fes",
36     "cis", "des", "dis",
37     "es",
38     "fis", "ges", "gis",
39     "as", "ais", "bes",
40     "c", "d", "e", "f", "g", "a", "b"
41 ]
42
43 NOTE_NAMES.sort(
44     key=len,
45     reverse=True
46 )
47
48 # 音符読取用の正規表現
49 #
50 # 読み取るもの:
51 #
52 #   音名
```

```

53 # オクターブ記号 , '
54 #
55 # 例:
56 #
57 # c
58 # c'
59 # fis,
60 # bes''
61 #
62 # 前後が英数字や _ の一部である場合は、
63 # 変数名やコマンド名の一部と見なして音符として読まない。
64 import re
65
66 NOTE_RE = re.compile(
67     r'(?![A-Za-z0-9_])'
68     r'(' +
69     '|'.join(NOTE_NAMES) +
70     r')'
71     r'([,]*)'
72     r'(?![A-Za-z])'
73 )
74
75
76 class Tokenizer:
77     """
78     LilyPond ソース文字列を Token 列へ変換するクラス。
79
80     self.text
81     入力文字列
82
83     self.pos
84     現在の読み取り位置
85
86     self.len
87     入力文字列の長さ
88     """
89
90     def __init__(self, text):
91         """
92         Tokenizer を初期化する。
93         """
94
95         self.text = text
96         self.pos = 0
97         self.len = len(text)
98
99     def peek(self, n=0):
100         """
101         現在位置から n 文字先を見る。
102
103         文字を読むだけで、self.pos は進めない。
104         範囲外なら空文字を返す。
105         """
106
107         p = self.pos + n
108
109         if p >= self.len:
110             return ""
111
112         return self.text[p]
113
114     def advance(self):
115         """
116         現在位置の文字を読み取り、1
117         self.pos を文字進める。1
118         """
119
120         ch = self.peek()
121
122         self.pos += 1
123
124         return ch
125
126     def read_comment(self):
127         """
128         コメントを読み取る。
129
130         % から行末直前までを
131         CommentToken として返す。
132         改行文字自体はここでは読まない。
133         """
134
135         buf = ""
136
137         while self.peek() not in ("", "\n"):
138             buf += self.advance()
139
140         return CommentToken(buf)
141

```

```

142 def read_command(self):
143     """
144     LilyPond コマンドを読み取る。
145
146     例:
147         \clef
148         \time
149         \major
150
151     バックスラッシュから始まり、
152     英数字または _ が続く部分を読む。
153     """
154
155     buf = self.advance()
156
157     while (
158         self.peek().isalnum()
159         or self.peek() == "_"
160     ):
161         buf += self.advance()
162
163     return CommandToken(buf)
164
165 def read_note(self):
166     """
167     音符を読み取る。
168
169     読み取る内容:
170
171         音名
172         オクターブ記号
173         後続の suffix
174
175     suffix には、音価・スラー・タイ・装飾などを含める。
176
177     例:
178         c'8(
179         fis,16~
180         bes4\\trill
181     """
182
183     m = NOTE_RE.match(
184         self.text,
185         self.pos
186     )
187
188     if not m:
189         return None
190
191     self.pos = m.end()
192
193     note = m.group(1)
194
195     octave = m.group(2)
196
197     suffix = ""
198
199     while (
200         self.peek()
201         and self.peek() not in
202         " \t\r\n<>"
203     ):
204         suffix += self.advance()
205
206     return NoteToken(
207         note,
208         octave,
209         suffix
210     )
211
212 def read_chord(self):
213     """
214     和音 < ... > を読み取る。
215
216     和音内の音符は NoteToken として、
217     空白は RawToken として、
218     その他の記号は SymbolToken として保持する。
219
220     和音閉じ記号 > の後に続く音価や記号は
221     chord の suffix として保持する。
222     """
223
224     self.advance() # '<'
225
226     items = []
227
228     while True:
229
230         if self.peek() == ">":

```

```

231         self.advance()
232         break
233
234     if self.peak().isspace():
235         items.append(
236             RawToken(
237                 self.advance()
238             )
239         )
240         continue
241
242     n = self.read_note()
243
244     if n:
245         items.append(n)
246         continue
247
248     items.append(
249         SymbolToken(
250             self.advance()
251         )
252     )
253
254     suffix = ""
255
256     while (
257         self.peak()
258         and self.peak() not in
259         "\t\r\n"
260     ):
261         suffix += self.advance()
262
263     return ChordToken(
264         items,
265         suffix
266     )
267
268 def tokenize(self):
269     """
270     入力文字列全体を Token 列へ変換する。
271
272     処理する主な構文:
273
274     \\relative { ... }
275     \\key c \\major
276     << ... >>
277     < ... >
278     コメント
279     コマンド
280     音符
281     その他の生文字列
282     """
283
284     result = []
285
286     while self.pos < self.len:
287
288         ch = self.peak()
289
290         # \\relative ブロック
291         if self.text.startswith(
292             "\\relative",
293             self.pos
294         ):
295             result.append(
296                 self.read_relative()
297             )
298             continue
299
300         # 並列ブロック開始
301         if self.text.startswith("<<", self.pos):
302             result.append(
303                 self.read_parallel()
304             )
305             continue
306
307         # 並列ブロック終了
308         if self.text.startswith(">>", self.pos):
309             self.pos += 2
310             result.append(SymbolToken(">>"))
311             continue
312
313         # 波括弧
314         if ch in "{}":
315             result.append(
316                 SymbolToken(
317                     self.advance()
318                 )
319             )

```

```

320         continue
321
322     # コメント
323     if ch == "%":
324         result.append(
325             self.read_comment()
326         )
327
328     continue
329
330     # 調指定
331     if self.text.startswith("\\key", self.pos):
332         result.append(self.read_key())
333         continue
334
335     # 一般コマンド
336     if ch == "\\":
337
338         if self.text.startswith(
339             "\\relative",
340             self.pos
341         ):
342             result.append(
343                 self.read_relative()
344             )
345             continue
346
347             result.append(
348                 self.read_command()
349             )
350             continue
351
352     # 和音
353     if ch == "<":
354
355         if self.peak(1) != "<":
356             result.append(
357                 self.read_chord()
358             )
359
360         continue
361
362     # 単音
363     n = self.read_note()
364
365     if n:
366         result.append(n)
367
368         continue
369
370     # どの構文にも該当しない文字はそのまま保持する
371     result.append(
372         RawToken(
373             self.advance()
374         )
375     )
376
377     return result
378
379 def read_key(self):
380     """
381     \\key コマンドを読み取る。
382
383     例:
384         \\key c \\major
385         \\key a \\minor
386
387     戻り値:
388         KeyToken(key, mode)
389     """
390
391     self.pos += len("\\key")
392
393     while self.peak().isspace():
394         self.advance()
395
396     pitch = self.read_pitch_literal()
397
398     if not pitch:
399         raise SyntaxError("key pitch expected")
400
401     key = pitch[0] + pitch[1]
402
403     while self.peak().isspace():
404         self.advance()
405
406     if self.peak() != "\\":
407         raise SyntaxError("key mode expected")
408

```

```

409         mode = self.read_command().text[1:]
410
411         return KeyToken(key, mode)
412
413     def read_pitch_literal(self):
414         """
415         音高リテラルを読み取る。
416
417         NoteToken は作らず、
418
419         音名 (, オクターブ記号)
420
421         のタブルを返す。
422
423         \\relative の anchor や
424         \\key の調名読み取りに使う。
425         """
426
427         m = NOTE_RE.match(
428             self.text,
429             self.pos
430         )
431
432         if not m:
433             return None
434
435         self.pos = m.end()
436
437         return (
438             m.group(1),
439             m.group(2)
440         )
441
442     def read_brace_block(self):
443         """
444         { ... } の中身を文字列として読み取る。
445
446         入れ子の { ... } にも対応する。
447
448         呼び出し時点では、
449         開き波括弧 { はすでに読み取り済みであることを前提にする。
450
451         戻り値には、外側の { } は含めない。
452         """
453
454         depth = 1
455
456         start = self.pos
457
458         while self.pos < self.len:
459
460             ch = self.advance()
461
462             if ch == "{":
463                 depth += 1
464
465             elif ch == "}":
466                 depth -= 1
467
468             if depth == 0:
469                 return self.text[start:self.pos - 1]
470
471             raise SyntaxError("missing }")
472
473     def read_relative(self):
474         """
475         \\relative ブロックを読み取る。
476
477         例:
478             \\relative c' { c d e }
479
480         処理内容:
481
482             anchor を読む
483             { ... } の中身を読む
484             中身を再度 Tokenizer にかける
485
486         戻り値:
487             RelativeBlock(anchor, child_tokens)
488         """
489
490         self.pos += len("\\relative")
491
492         while self.peek().isspace():
493             self.advance()
494
495         pitch = self.read_pitch_literal()
496
497         if not pitch:

```



```

498         raise SyntaxError(
499             "relative anchor expected"
500         )
501
502     anchor = pitch[0] + pitch[1]
503
504     while self.peak().isspace():
505         self.advance()
506
507     if self.peak() != "{":
508         raise SyntaxError(
509             "{ expected"
510         )
511
512     self.advance()
513
514     body = self.read_brace_block()
515
516     child_tokens = Tokenizer(
517         body
518     ).tokenize()
519
520     return RelativeBlock(
521         anchor,
522         child_tokens
523     )
524
525 def read_parallel(self):
526     """
527     並列ブロック << ... >> を読み取る。
528
529     対応する主な形:
530
531         <<
532         { voice1 }
533         { voice2 }
534         >>
535
536     および
537
538         <<
539         \\new Voice { voice1 }
540         \\new Voice { voice2 }
541         >>
542
543     各 voice を個別に tokenize し、
544     ParallelBlock として返す。
545     """
546
547     self.pos += 2 # skip <<
548
549     voices = []
550
551     while self.pos < self.len:
552
553         while self.peak().isspace():
554             self.advance()
555
556         if self.text.startswith(">>", self.pos):
557             self.pos += 2
558             break
559
560         # { ... } 形式の voice
561         if self.peak() == "{":
562             self.advance()
563             body = self.read_brace_block()
564
565             voices.append(
566                 Tokenizer(body).tokenize()
567             )
568             continue
569
570         # \\new Voice { ... } 形式の voice
571         start = self.pos
572
573         if self.text.startswith("\\new", self.pos):
574
575             prefix = ""
576
577             while (
578                 self.pos < self.len
579                 and self.peak() != "{"
580             ):
581                 prefix += self.advance()
582
583             if self.peak() != "{":
584                 raise SyntaxError(
585                     "missing { after \\new Voice"
586                 )

```

```
587         self.advance()
588         body = self.read_brace_block()
589
590         voice_tokens = (
591             Tokenizer(prefix).tokenize()
592             + [SymbolToken("{")]
593             + Tokenizer(body).tokenize()
594             + [SymbolToken("}")]
595         )
596
597         voices.append(voice_tokens)
598         continue
599
600     # 想定外の文字は読み飛ばす
601     self.advance()
602
603
604 return ParallelBlock(voices)
```

Listing 8.1 字句解析 tokenizer.py

第 9 章

音名と半音の理論計算

9.1 music_theory.py の役割

NOTE_TO_SEMITONE は音名を 0 から 11 のピッチクラスへ変換する。transpose_note_name_by_interval() は、半音移動量だけでなく音名文字の移動量も受け取り、新しい文字と臨時記号を別々に決める。

例えば C から A への移調では半音移動量は -3、文字移動量は 5 である。したがって C は A、D は B、E は C♯ というように、音程の文字関係を保った綴りが得られる。生成結果がダブルシャープまたはダブルフラットになる場合は、実用性を優先して代表的な綴りへ簡略化する。

```
1 # music_theory.py
2 #
3 # 音名と半音値の相互変換を担当する。
4 #
5 # 主な責務:
6 #
7 #     音名 → 半音値
8 #     半音値 → 音名
9 #     移調後の綴り決定
10 #     シャープ・フラット処理
11 #
12 # やの相対音高は扱わず、MIDI LilyPond
13 # 純粋な音楽理論部分のみを担当する。
14 # 音高テーブル
15 # 音名 → 半音値変換テーブル
16 #
17 # 例:
18 #
19 #     c    -> 0
20 #     fis  -> 6
21 #     bes  -> 10
22 #     bis  -> 0
23 #
24 # ダブルシャープ・ダブルフラットも対応。
25 NOTE_TO_SEMITONE = {
26     "ces": 11,
27     "c": 0,
28     "cis": 1,
29     "des": 1,
30     "d": 2,
31     "dis": 3,
32     "ees": 3,
33     "es": 3,
34     "e": 4,
35     "eis": 5,
36     "fes": 4,
37     "f": 5,
38     "fis": 6,
39     "ges": 6,
40     "g": 7,
41     "gis": 8,
42     "ases": 7,
43     "as": 8,
44     "a": 9,
45     "ais": 10,
46     "beses": 8,
47     "bes": 10,
48     "b": 11,
49     "bis": 0,
50 }
51 NOTE_TO_SEMITONE.update({
52     # "bis": 0,
53     "cisis": 2,
54     "disis": 5,
```

```

55     "eisis": 6,
56     "fisis": 7,
57     "gisis": 9,
58     "aisis": 11,
59
60     "ceses": 10,
61     "deses": 0,
62     "eeses": 2,
63     "fesese": 3,
64     "geses": 5,
65     "aseses": 6,
66     "beses": 8,
67 })
68 # 音名文字 (c,d,e...)のインデックス
69 #
70 # 文字移調量 (letter_shift)計算で使用する。
71 LETTER_INDEX = {
72     "c": 0,
73     "d": 1,
74     "e": 2,
75     "f": 3,
76     "g": 4,
77     "a": 5,
78     "b": 6,
79 }
80 # 半音値から代表的なシャープ系表記へ変換する。
81 #
82 # 例:
83 #
84 #     3 -> dis
85 #     8 -> gis
86 COMMON_PC_TO_NOTE = {
87     0: "c",
88     1: "cis",
89     2: "d",
90     3: "dis",
91     4: "e",
92     5: "f",
93     6: "fis",
94     7: "g",
95     8: "gis",
96     9: "a",
97     10: "ais",
98     11: "b",
99 }
100 # 半音値から代表的なフラット系表記へ変換する。
101 #
102 # 例:
103 #
104 #     3 -> es
105 #     10 -> bes
106 SEMITONE_TO_NOTE = {
107     0: "c",
108     1: "cis",
109     2: "d",
110     3: "es",
111     4: "e",
112     5: "f",
113     6: "fis",
114     7: "g",
115     8: "as",
116     9: "a",
117     10: "bes",
118     11: "b",
119 }
120 # 自然音 (c,d,e,f,g,a,b)の半音値。
121 #
122 # 臨時記号 (accidental)計算の基準として使用する。
123 LETTER_TO_NATURAL_PC = {
124     "c": 0,
125     "d": 2,
126     "e": 4,
127     "f": 5,
128     "g": 7,
129     "a": 9,
130     "b": 11,
131 }
132 # 音名の並び順。
133 #
134 # 文字移調 (letter_shift)の計算で使用する。
135 LETTERS = ["c", "d", "e", "f", "g", "a", "b"]
136
137
138 def transpose_note_name_by_interval(
139     old_note,
140     semitone_shift,
141     letter_shift
142 ):
143     """

```

```

144     音名を移調する。
145
146     入力:
147
148         old_note
149         元の音名
150
151         semitone_shift
152         半音移動量
153
154         letter_shift
155         音名文字の移動量
156
157     例:
158
159         c -> a
160
161     の場合
162
163         semitone_shift = -3
164         letter_shift = 5
165
166     を使い、
167
168         c -> a
169         d -> b
170         e -> cis
171
172     のような綴りを決定する。
173     """
174     old_letter = old_note[0]
175
176     new_letter = LETTERS[
177         (
178             LETTERS.index(old_letter)
179             + letter_shift
180         ) % 7
181     ]
182
183     old_pc = NOTE_TO_SEMITONE[old_note]
184     new_pc = (old_pc + semitone_shift) % 12
185
186     natural_pc = LETTER_TO_NATURAL_PC[new_letter]
187
188     acc = (new_pc - natural_pc) % 12
189
190     if acc > 6:
191         acc -= 12
192
193     note = note_from_letter_acc(
194         new_letter,
195         acc
196     )
197
198     return simplify_spelling(note)
199
200
201 def transpose_pitch(
202     note,
203     shift
204 ):
205     """
206     音名を半音数だけ移調する。
207
208     綴りは考慮せず、
209     SEMITONE_TO_NOTE を用いて
210     単純変換する。
211
212     主に \\relative 外の処理で使用。
213     """
214     pc = NOTE_TO_SEMITONE[note]
215
216     return SEMITONE_TO_NOTE[
217         (pc + shift) % 12
218     ]
219
220
221 def simplify_spelling(note):
222     """
223     音名を簡略表記へ変換する。
224
225     例:
226
227         disis -> f
228         beses -> gis
229
230     ダブルシャープ・ダブルフラットを
231     避けるために使用する。
232     """

```

```

233 pc = NOTE_TO_SEMITONE[note]
234
235 # ダブルシャープ・ダブルフラットを避ける
236 if "isis" in note or "eses" in note:
237     return COMMON_PC_TO_NOTE[pc]
238
239 return note
240
241
242 def note_from_letter_acc(letter, acc):
243     """
244     音名文字と臨時記号数から
245     音名を生成する。LilyPond
246
247     例:
248
249         ("c", 0)  -> c
250         ("c", 1)  -> cis
251         ("c", 2)  -> cisis
252         ("b", -1) -> bes
253         ("a", -2) -> ases
254     """
255     if acc == 0:
256         return letter
257
258     if acc == 1:
259         return letter + "is"
260
261     if acc == 2:
262         return letter + "isis"
263
264     if acc == -1:
265         if letter == "a":
266             return "as"
267         if letter == "e":
268             return "es"
269         if letter == "b":
270             return "bes"
271         return letter + "es"
272
273     if acc == -2:
274         if letter == "a":
275             return "ases"
276         if letter == "e":
277             return "eses"
278         if letter == "b":
279             return "beses"
280         return letter + "eses"
281
282     raise ValueError(
283         f"unsupported accidental: {letter}, {acc}"
284     )
285
286
287 def split_note_name(note):
288     """
289     音名を LilyPond
290
291     文字部分
292     臨時記号数
293
294     に分解する。
295
296     例:
297
298         fis  -> ("f", 1)
299         bes  -> ("b", -1)
300         ases -> ("a", -2)
301         c    -> ("c", 0)
302     """
303     letter = note[0]
304     rest = note[1:]
305
306     if rest == "":
307         acc = 0
308
309     elif rest == "is":
310         acc = 1
311
312     elif rest == "isis":
313         acc = 2
314
315     elif note == "as":
316         letter = "a"
317         acc = -1
318
319     elif note == "ases":
320         letter = "a"
321         acc = -2

```

```
322
323     elif note == "es":
324         letter = "e"
325         acc = -1
326
327     elif note == "eses":
328         letter = "e"
329         acc = -2
330
331     elif note == "bes":
332         letter = "b"
333         acc = -1
334
335     elif note == "beses":
336         letter = "b"
337         acc = -2
338
339     elif rest == "es":
340         acc = -1
341
342     elif rest == "eses":
343         acc = -2
344
345     else:
346         raise ValueError(
347             f"bad accidental: {note}"
348         )
349
350     return letter, acc
351
352
353 def note_base_midi(note):
354     """
355     音名の基準値を返す。MIDI
356
357     c を MIDI 60 として計算する。
358
359     例:
360
361         c    -> 60
362         d    -> 62
363         fis  -> 66
364         bes  -> 70
365
366     オクターブは含まない。
367     """
368     letter, acc = split_note_name(note)
369
370     return (
371         60
372         + LETTER_TO_NATURAL_PC[letter]
373         + acc
374     )
```

Listing 9.1 音楽理論計算 music_theory.py

第 10 章

絶対音高とオクターブ

10.1 pitch.py の役割

`split_pitch()` は `fis`, を音名 `fis` とオクターブ記号, へ分ける。音名の後ろには' または, だけを許可し、上下記号の混在を拒否する。`parse_pitch()` は `C` を MIDI 値 60 の基準とし、アポストロフィーの個数からカンマの個数を引いた値に 12 を掛けてオクターブ差を加える。

`PitchPos` は音名、オクターブ、音名文字、MIDI 値をまとめて保持する。`midi_to_absolute_lily()` は計算後の MIDI 値を絶対 LilyPond 表記へ戻し、`\relative` の新しい基準音生成に使用される。

```
1 # pitch.py
2 #
3 # LilyPond の音高表記と内部音高 (MIDI)との
4 # 相互変換を担当するモジュール。
5 #
6 # 例:
7 #
8 #   c      -> MIDI 60
9 #   c'     -> MIDI 72
10 #   fis,   -> MIDI 54
11 #
12 # Relative 解決のための PitchPos クラスも定義する。
13
14 from dataclasses import dataclass
15
16
17 @dataclass
18 class PitchPos:
19     """
20     絶対音高を保持する内部データ。
21
22     note
23         音名 LilyPond
24         例: c, fis, bes
25
26     octave
27         c を基準としたオクターブ番号
28
29     letter
30         音名の文字部分
31         例: c,d,e,f,g,a,b
32
33     midi
34         音高番号 MIDI
35     """
36     note: str
37     octave: int
38     letter: str
39     midi: int
40
41
42 from tokenizer import NOTE_NAMES
43 from music_theory import (
44     NOTE_TO_SEMITONE,
45     SEMITONE_TO_NOTE,
46     note_base_midi,
47 )
48
49
50 def split_pitch(pitch):
51     """
52     音高文字列を LilyPond
53
54     音名
```

```

55     オクターブ記号
56
57     に分解する。
58
59     例:
60
61     c'  -> ("c", "'")
62     fis -> ("fis", ",")
63     bes -> ("bes", "")
64     """
65     for name in NOTE_NAMES:
66
67         if pitch.startswith(name):
68             marks = pitch[len(name):]
69
70             # 音名の後ろにはオクターブ記号だけを許可する。LilyPond
71             # startswith()だけでは bis を、bcisis を cis と誤読するため、
72             # 残りの文字列も必ず検査する。
73             if any(ch not in "'", " for ch in marks):
74                 continue
75
76             # 上下を同時に指定する表記は曖昧なので受け付けない。
77             if "'" in marks and "," in marks:
78                 raise ValueError(f"mixed octave marks: {pitch}")
79
80             return (
81                 name,
82                 marks
83             )
84
85     raise ValueError(
86         f"bad pitch: {pitch}"
87     )
88
89
90 def parse_pitch(pitch):
91     """
92     音高文字列を LilyPond
93     音高へ変換する。MIDI
94
95     例:
96
97     c      -> 60
98     c'     -> 72
99     fis,   -> 54
100     """
101     note, octave = split_pitch(
102         pitch
103     )
104
105     midi = 60 + NOTE_TO_SEMITONE[
106         note
107     ]
108
109     midi += (
110         octave.count("'")
111         - octave.count(",")
112     ) * 12
113
114     return midi
115
116
117 def parse_absolute_pitch_pos(pitch):
118     """
119     音高文字列から LilyPond
120     PitchPos を生成する。
121
122     例:
123
124     c'
125     ↓
126
127     PitchPos(
128         note='c',
129         octave=1,
130         ...
131     )
132     """
133     note, marks = split_pitch(pitch)
134
135     octave = (
136         marks.count("'")
137         - marks.count(",")
138     )
139
140     return make_pos(note, octave)
141
142 def make_pos(note, octave):
143     # 以前は dict を返していたが、

```

```

144     # 現在は PitchPos を返す。
145     """
146     音名とオクターブ番号から
147     PitchPos を生成する。
148
149     Relative 解決処理の
150     基本データ生成関数。
151     """
152     midi = (
153         note_base_midi(note)
154         + octave * 12
155     )
156
157     return PitchPos(
158         note=note,
159         octave=octave,
160         letter=note_letter(note),
161         midi=midi,
162     )
163
164
165 def note_letter(note):
166     """
167     音名から文字部分だけを取り出す。
168
169     例:
170
171         fis -> f
172         bes -> b
173         c   -> c
174     """
175     return note[0]
176
177
178 def midi_to_absolute_lily(midi):
179     """
180     音高を MIDI
181     絶対表記へ変換する。 LilyPond
182
183     例:
184
185         60 -> c
186         72 -> c'
187         54 -> fis,
188
189     戻り値は
190     \\relative を使わない
191     絶対表記である。
192     """
193     pc = midi % 12
194     note = SEMITONE_TO_NOTE[pc]
195
196     octave_count = (midi - (60 + pc)) // 12
197
198     if octave_count > 0:
199         octave = "" * octave_count
200     elif octave_count < 0:
201         octave = "," * (-octave_count)
202     else:
203         octave = ""
204
205     return note + octave

```

Listing 10.1 音高変換 pitch.py

第 11 章

相対音高の解決

11.1 relative.py の役割

LilyPond の `\relative` では、各音符の実際のオクターブが直前音との関係で決まる。`RelativeResolver` は移調前と移調後の直前音をそれぞれ保持する。

単音を処理するときは、まず `resolve_relative_pitch()` で元の絶対音高を求め、半音移動量を加え、移調後の直前音から必要な¹ または、² を再計算する。これにより、音名だけを置換する場合に起こる意図しないオクターブ移動を避ける。

和音では構成音を順に解決する一方、和音の次の単音へ引き継ぐ基準には和音の第 1 音を使う。複数声部では `Resolver` を複製し、各声部が別の直前音状態を持つようにする。

```
1 import copy
2 from tokens import *
3 from pitch import (
4     parse_absolute_pitch_pos,
5     make_pos,
6     note_letter,
7 )
8 from music_theory import (
9     LETTER_INDEX,
10    note_base_midi,
11    transpose_note_name_by_interval,
12 )
13
14 class RelativeResolver:
15     """
16     \\relative ブロック内の音高解決を担当する。
17
18     LilyPond の相対音高規則を解釈し、
19
20         元の音高
21         ↓
22         移調後の絶対音高
23         ↓
24         相対表記 LilyPond
25
26     に変換する。
27
28     RelativeResolver
29     \\relative 内の直前音を覚えながら
30     NoteToken / ChordToken を移調する
31     """
32     def __init__(self, anchor, letter_shift):
33         self.prev_old_pos = parse_absolute_pitch_pos(anchor)
34         self.prev_new_pos = None
35         self.current_key = "c"
36         self.letter_shift = letter_shift
37
38     def clone(self):
39         """
40         Resolver の複製を作る。
41
42         << ... >> や ParallelBlock の各声部を
43         独立に処理するために使用する。
44         """
45         return copy.deepcopy(self)
46
47     def transpose_note(self, note_token, shift):
48         """
49         単音 (NoteToken)を移調する。
50
```

```

51     相対表記から元の絶対音高を求め、
52     指定半音数だけ移調し、
53     の相対表記へ戻す。LilyPond
54     """
55     old_pos = resolve_relative_pitch(
56         note_token,
57         self.prev_old_pos
58     )
59
60     new_midi = old_pos.midi + shift
61
62     new_note = transpose_note_name_by_interval(
63         old_pos.note,
64         shift,
65         self.letter_shift
66     )
67
68     marks, new_pos = midi_to_lilypond_relative_with_note(
69         self.prev_new_pos,
70         new_midi,
71         new_note
72     )
73
74     note_token.note = new_note
75     note_token.octave = marks
76
77     self.prev_old_pos = old_pos
78     self.prev_new_pos = new_pos
79
80 def transpose_chord(self, chord, shift):
81     """
82     和音 (ChordToken)を移調する。
83
84     和音内の各音を順番に処理し、
85     の相対表記を維持したまま LilyPond
86     移調結果へ書き換える。
87     """
88     first_old_pos = None
89     first_new_pos = None
90
91     chord_old_prev = self.prev_old_pos
92     chord_new_prev = self.prev_new_pos
93
94     for item in chord.items:
95
96         if not isinstance(item, NoteToken):
97             continue
98
99         old_pos = resolve_relative_pitch(
100             item,
101             chord_old_prev
102         )
103
104         new_midi = old_pos.midi + shift
105
106         new_note = transpose_note_name_by_interval(
107             old_pos.note,
108             shift,
109             self.letter_shift
110         )
111
112         marks, new_pos = midi_to_lilypond_relative_with_note(
113             chord_new_prev,
114             new_midi,
115             new_note
116         )
117
118         item.note = new_note
119         item.octave = marks
120
121         chord_old_prev = old_pos
122         chord_new_prev = new_pos
123
124         if first_old_pos is None:
125             first_old_pos = old_pos
126             first_new_pos = new_pos
127
128         if first_old_pos is not None:
129             self.prev_old_pos = first_old_pos
130             self.prev_new_pos = first_new_pos
131
132
133 def resolve_relative_pitch(note_token, prev_pos):
134     """
135     NoteToken を絶対音高へ展開する。
136
137     LilyPond の \\relative 規則に従い、
138     直前音 (prev_pos)から最も近い音域を選ぶ。
139     """

```

```

140     note = note_token.note
141
142     octave = lilypond_inferred_octave(
143         prev_pos,
144         note
145     )
146
147     octave += (
148         note_token.octave.count("")
149         - note_token.octave.count(",")
150     )
151
152     return make_pos(note, octave)
153
154
155 def lilypond_inferred_octave(prev_pos, note):
156     """
157     LilyPond の相対音高規則を実装する。
158
159     prev_pos に最も近い音域の note を選び、
160     推定オクターブ番号を返す。
161     """
162     prev_step = (
163         prev_pos.octave * 7
164         + LETTER_INDEX[prev_pos.letter]
165     )
166
167     letter = note_letter(note)
168     letter_index = LETTER_INDEX[letter]
169
170     candidates = []
171
172     for octave in range(
173         prev_pos.octave - 4,
174         prev_pos.octave + 5
175     ):
176         step = octave * 7 + letter_index
177         diff = step - prev_step
178
179         candidates.append(
180             (octave, diff)
181         )
182
183     octave, diff = min(
184         candidates,
185         key=lambda x: (
186             abs(x[1]),
187             x[1]
188         )
189     )
190
191     return octave
192
193
194 def midi_to_lilypond_relative_with_note(
195     prev_pos,
196     midi,
197     note
198 ):
199     """
200     絶対音高 (MIDI)を
201     相対表記へ変換する。LilyPond
202
203     戻り値:
204         (""', PitchPos(...))
205
206     のような
207     オクターブ記号
208     新しい PitchPos
209     の組を返す。
210     """
211     abs_octave = (
212         midi
213         - note_base_midi(note)
214     ) // 12
215
216     inferred_octave = lilypond_inferred_octave(
217         prev_pos,
218         note
219     )
220
221     diff = abs_octave - inferred_octave
222
223     if diff > 0:
224         marks = "" * diff
225     elif diff < 0:
226         marks = "," * (-diff)
227     else:
228         marks = ""

```

```
229 |  
230 | return marks, make_pos(note, abs_octave)
```

Listing 11.1 相对音高处理 relative.py

第 12 章

Token 列の移調

12.1 transpose.py の役割

`walk_tokens()` は Token 列を再帰的に走査し、種類に応じて処理を振り分ける。`RelativeBlock` では専用 Resolver を作り、通常の単音・和音では現在 Resolver があるかどうかによって相対表記用または単純移調用の処理を選ぶ。

`KeyToken` はモードを維持したまま主音を移調する。並列ブロックでは Resolver を複製し、一つの声部の終端音が別の声部の開始音へ影響しないようにする。

```

1  # transpose.py
2  '''
3  transpose.py
4  列を歩く Token
5  '''
6  import copy
7  from tokens import *
8  from music_theory import *
9  from pitch import *
10 from relative import RelativeResolver
11
12
13 def letter_shift_between(src, dst):
14     """
15     音名の文字移動量を求める。
16
17     例:
18         c -> a : 5
19         c -> d : 1
20         g -> d' : 4
21
22     半音数ではなく、
23     c, d, e, f, g, a, b の文字位置の差を返す。
24     """
25     src_note, _ = split_pitch(src)
26     dst_note, _ = split_pitch(dst)
27
28     src_letter = src_note[0]
29     dst_letter = dst_note[0]
30
31     return (
32         LETTERS.index(dst_letter)
33         - LETTERS.index(src_letter)
34     ) % 7
35
36
37
38 def transpose_relative_block(block, shift, letter_shift):
39     """
40     RelativeBlock 全体を移調する。
41
42     anchor を移調し、
43     新しい RelativeResolver を作成して
44     ブロック内部を再帰的に処理する。
45     """
46     old_anchor = block.anchor
47
48     resolver = RelativeResolver(
49         old_anchor,
50         letter_shift
51     )
52
53     block.anchor = midi_to_absolute_lily(
54         parse_pitch(old_anchor) + shift

```

```

55     )
56
57     resolver.prev_new_pos = parse_absolute_pitch_pos(
58         block.anchor
59     )
60
61     walk_tokens(
62         block.tokens,
63         resolver,
64         shift,
65         letter_shift
66     )
67
68
69 def walk_tokens(tokens, resolver, shift, letter_shift):
70     # << ... >> を検出したら
71     # resolver をコピーして独立に処理する
72     """
73     列を再帰的に走査する。Token
74
75     処理対象:
76
77         RelativeBlock
78         NoteToken
79         ChordToken
80         KeyToken
81         ParallelBlock
82         << >> ブロック
83
84     を見つけると適切な移調処理を行う。
85     """
86     i = 0
87
88     while i < len(tokens):
89
90         t = tokens[i]
91
92         if (
93             isinstance(t, SymbolToken)
94             and t.text == "<<"
95         ):
96             depth = 1
97             j = i + 1
98
99             while j < len(tokens):
100
101                 x = tokens[j]
102
103                 if (
104                     isinstance(x, SymbolToken)
105                     and x.text == "<<"
106                 ):
107                     depth += 1
108
109                 elif (
110                     isinstance(x, SymbolToken)
111                     and x.text == ">>"
112                 ):
113                     depth -= 1
114
115                     if depth == 0:
116                         break
117
118                 j += 1
119
120             inner = tokens[i + 1:j]
121
122             if resolver:
123                 # local_resolver = resolver.clone()
124                 local_resolver = copy.deepcopy(
125                     resolver
126                 )
127             else:
128                 local_resolver = None
129
130             walk_tokens(
131                 inner,
132                 local_resolver,
133                 shift,
134                 letter_shift
135             )
136
137             i = j + 1
138             continue
139
140     if isinstance(t, RelativeBlock):
141
142         transpose_relative_block(
143             t,

```

```

144         shift,
145         letter_shift
146     )
147
148     elif isinstance(t, NoteToken):
149
150         if resolver:
151             resolver.transpose_note(
152                 t,
153                 shift
154             )
155         else:
156             t.note = transpose_pitch(
157                 t.note,
158                 shift
159             )
160
161     elif isinstance(t, ChordToken):
162
163         if resolver:
164             resolver.transpose_chord(
165                 t,
166                 shift
167             )
168         else:
169             for item in t.items:
170                 if isinstance(item, NoteToken):
171                     item.note = transpose_pitch(
172                         item.note,
173                         shift
174                     )
175
176     elif isinstance(t, KeyToken):
177
178         old_key = t.key
179
180         new_key = transpose_pitch(
181             old_key,
182             shift
183         )
184
185         t.key = new_key
186
187         if resolver:
188             resolver.current_key = new_key
189
190     elif isinstance(t, ParallelBlock):
191
192         for voice in t.voices:
193
194             if resolver:
195                 # local_resolver = resolver.clone()
196                 local_resolver = copy.deepcopy(
197                     resolver
198                 )
199             else:
200                 local_resolver = None
201
202             walk_tokens(
203                 voice,
204                 local_resolver,
205                 shift,
206                 letter_shift
207             )
208
209         i += 1
210
211
212 def transpose_tokens(tokens, shift, letter_shift):
213     """
214     列全体を移調する。Token
215
216     移調処理のエントリポイント。
217     """
218     walk_tokens(
219         tokens,
220         None,
221         shift,
222         letter_shift
223     )

```

Listing 12.1 Token 走査と移調 transpose.py

第 13 章

LilyPond 文字列への復元

13.1 writer.py の役割

`write_token()` は `Token` の種類ごとに LilyPond 表記を組み立てる。`RawToken`、`CommentToken`、`CommandToken` は内容を保持し、`NoteToken` は音名、再計算済みオクターブ記号、`suffix` を連結する。入れ子を持つ `Token` は子 `Token` を再帰的に文字列化する。

`Tokenizer` と `Writer` を対にすることで、移調対象以外の情報を可能な限り保存したまま、必要な音高部分だけを書き換える。

```

1  # writer.py
2  #
3  # Token オブジェクトを LilyPond ソース文字列へ戻す。
4  #
5  # tokenizer.py が
6  #
7  #   文字列 → Token
8  #
9  # を担当するのに対して、このファイルは
10 #
11 #   Token → 文字列
12 #
13 # を担当する。
14
15 from tokens import *
16
17
18 def write_token(t):
19     """
20     つの 1 Token を LilyPond ソース文字列へ変換する。
21
22     RawToken, NoteToken, ChordToken, RelativeBlock など、
23     Token の種類ごとに元の LilyPond 表記へ戻す。
24     """
25
26     if isinstance(t, RawToken):
27         # 空白・改行・その他の生文字列はそのまま出力する。
28         return t.text
29
30     elif isinstance(t, CommandToken):
31         # \clef, \time などのコマンドはそのまま出力する。
32         return t.text
33
34     elif isinstance(t, KeyToken):
35         # KeyToken を \key c \major の形へ戻す。
36         return (
37             "\\key "
38             + t.key
39             + " \\ "
40             + t.mode
41         )
42
43     elif isinstance(t, CommentToken):
44         # コメントはそのまま出力する。
45         return t.text
46
47     elif isinstance(t, SymbolToken):
48         # { }, <, > などの記号はそのまま出力する。
49         return t.text
50
51     elif isinstance(t, NoteToken):
52         # 単音を note + octave + suffix の形で出力する。
53         #
54         # 例:

```

```

55     #     note    = c
56     #     octave = '
57     #     suffix = 8(
58     #
59     #     => c'8(
60     return (
61         t.note
62         + t.octave
63         + t.suffix
64     )
65
66 elif isinstance(t, ChordToken):
67     # 和音 < ... > を出力する。
68     #
69     # 和音内部の NoteToken, RawToken, SymbolToken を
70     # 順番に文字列化してから、最後に suffix を付ける。
71     s = "<"
72
73     for item in t.items:
74         if isinstance(item, NoteToken):
75             s += (
76                 item.note
77                 + item.octave
78                 + item.suffix
79             )
80
81         elif isinstance(item, SymbolToken):
82             s += item.text
83
84         elif isinstance(item, RawToken):
85             s += item.text
86
87     s += ">"
88     s += t.suffix
89
90     return s
91
92 elif isinstance(t, RelativeBlock):
93     # \relative ブロックを出力する。
94     #
95     # 内部 tokens をすべて文字列化して body を作り、
96     # \relative anchor { ... } の形に戻す。
97     body = "".join(
98         write_token(x)
99         for x in t.tokens
100     )
101
102     return (
103         "\\relative "
104         + t.anchor
105         + " {\n"
106         + body
107         + "\n}"
108     )
109
110 elif isinstance(t, ParallelBlock):
111     # << ... >> 形式の並列ブロックを出力する。
112     #
113     # 各 voice を { ... } で囲んで書き戻す。
114     return (
115         "<<"
116         + "".join(
117             "{"
118             + "".join(
119                 write_token(x)
120                 for x in voice
121             )
122             + "}"
123             for voice in t.voices
124         )
125         + ">>"
126     )
127
128 # 未対応の Token が来た場合はエラーにする。
129 raise TypeError(type(t))

```

Listing 13.1 LilyPond 出力 writer.py

第 14 章

保守と検証

14.1 変更時に確認する事項

音名や対応構文を追加するときは、一つのテーブルだけでなく、Tokenizer、音楽理論テーブル、音高変換、Writer の整合性を確認する。少なくとも次の組合せを検証する。

- 上方向、下方向、1 オクターブをまたぐ移調
- シャープ系、フラット系、異名同音
- 単音、和音、調号
- `\relative` 内の上行・下行・明示的オクターブ指示
- 並列声部
- コメント、音価、スラー、タイなどの保持
- 不正な音名および `'` と `,` の混在の拒否

最終検証は文字列比較だけで終えず、生成したソースを LilyPond でコンパイルし、PDF 上の音高と譜面も確認する。